

Sessions 8 & 9

Computing, Robotics and Bionics

Sergio Sama Rubio

Sant Cugat del Vallès, 5 & 8-Oct-2026

Announcements

- Géron Part I companion (PDF) is on Drive — Chapters 1 and 2 are these two sessions. Read §1 before Session 8 if you can
- Bring the book and have the ageron/handson-mlp notebooks open — the PyTorch edition's own repository. The labs are Géron's Chapter 1 and Chapter 2 notebooks
- Miniproject 1 is released at the end of Session 9, and its brief follows the Chapter 2 workflow step for step

Previously on CRB1

- Session 7: LLMs — how they work — and the AI coding assistants you will be using all term
- Sessions 2–6 completed the five refreshers: the Python stack, linear algebra, calculus, image processing, and the AI tools
- That is the whole toolbox. From today the course is machine learning itself, following Géron Part I

Questions?

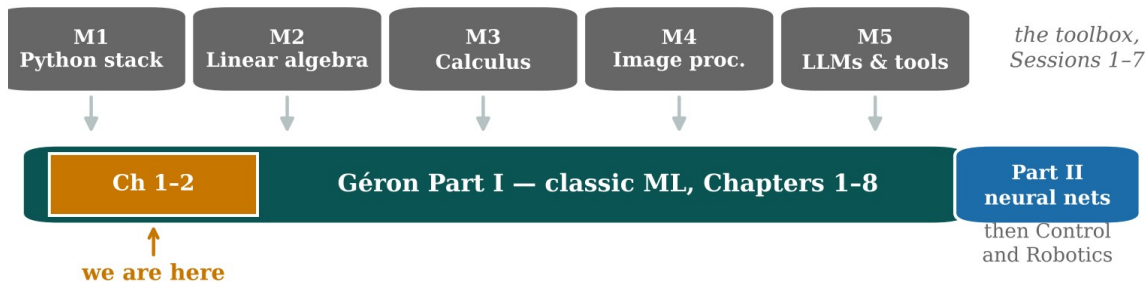
Today's Agenda

- Session 8 — Géron Chapter 1: The Machine Learning Landscape
 - what learning is, why not rules, the training signals, what goes wrong in the data, capacity and overfitting, honest validation, bias–variance, No Free Lunch
- Session 9 — Géron Chapter 2: End-to-End Machine Learning Project
 - the workflow, which error norm, a trustworthy test set, data snooping, preparing the data, selecting a model
- Labs — Géron's own Chapter 1 and Chapter 2 notebooks, run in Colab
- Kahoot

Part I Begins

Géron Part I companion (PDF) — where these two sessions sit

- The five refreshers were the toolbox: NumPy and Pandas for the data, linear algebra for projection and the SVD, calculus for the gradient, image processing for convolution, the AI assistants for the code
- Part I is classic machine learning end to end — Chapters 1–8, one model family at a time. Chapters 1 and 2 come first because every later chapter assumes them

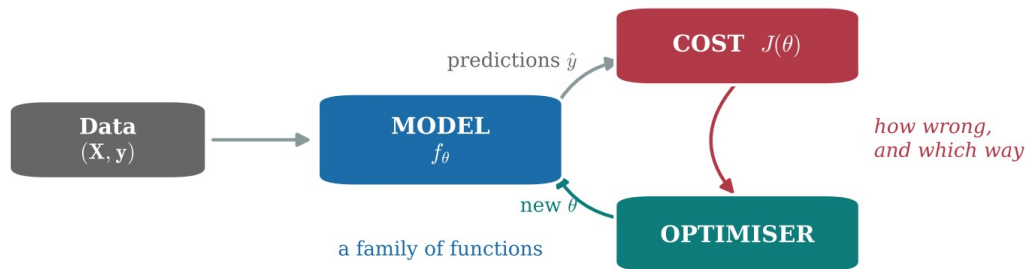


What Machine Learning Is

Part I companion (PDF), §1 — experience in place of rules

- A program learns from experience E at task T measured by performance P if its P on T improves with E . The rules are not written down; they are fitted
- Three ingredients, and every chapter from 4 onward is a different choice of them: a model, a cost that scores how badly it fits, and an optimiser that minimises that cost — gradient descent, from Module 3

Every model in Part I is one choice of these three



Chapter 4 names them for linear models; every later chapter swaps one and keeps the loop

Why Learn, Not Code Rules

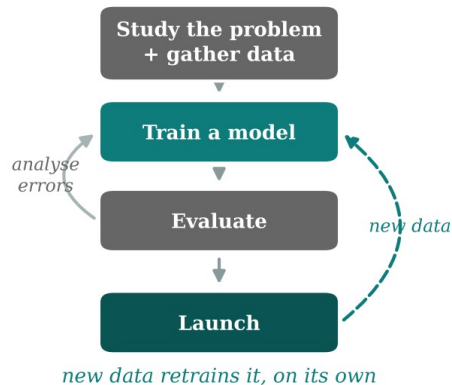
Géron Ch 1, Why Use Machine Learning? — what rules cannot do

- Written as rules, a spam filter becomes a long list of patterns someone has to maintain. Trained from examples it finds the predictive words itself, and the program is shorter and usually more accurate
- When the world moves — spammers write 'For U' instead of '4U' — the rule-based filter needs a new rule, while the learned one retrains on fresh data. It also works where no algorithm is known at all

Rules, written by hand



Rules, learned from data



A Taxonomy of Learning

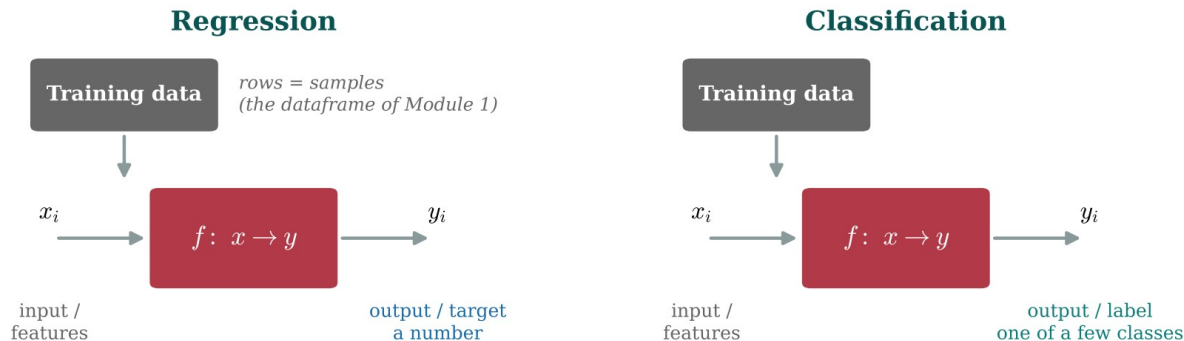
Part I companion (PDF), §1.1 — four kinds of training signal

- Supervised: every example carries a label. A continuous label makes it regression (a tumour's diameter); a discrete one makes it classification (benign vs malignant)
- Unsupervised: no labels — clustering patients by expression profile, dimensionality reduction (Chapter 7), density estimation, flagging an abnormal ECG
- Self- and semi-supervised build labels out of the data itself; reinforcement learns a policy from a reward signal — a closed loop, and conceptually the Control unit

Two Supervised Tasks

Géron Ch 1, Supervised Learning — the same machine, two kinds of output

- Both learn a function from features to output. The label decides which task it is: a continuous target makes it regression, a label from a finite set makes it classification
- The vocabulary is the dataframe of Module 1 — rows are samples, columns are features, and one column is the thing you are trying to predict



Two Orthogonal Axes

Part I companion (PDF), §1.1 — how a learner is fitted, and how it generalises

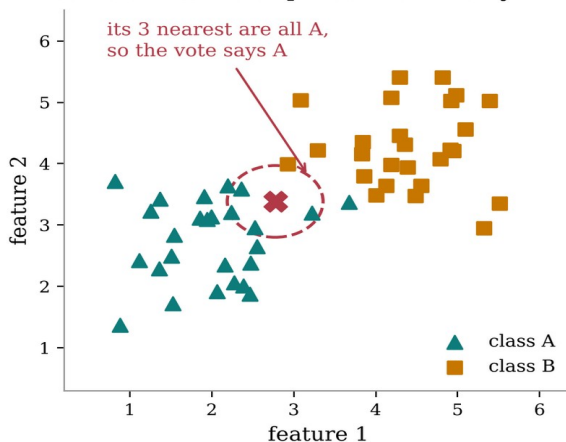
- Batch vs online: a batch learner trains once on all the data; an online learner updates incrementally as data streams in — the distinction that decides whether a ward monitor can keep learning
- Instance- vs model-based: whether the learner keeps the examples and compares, or fits parameters and generalises through the fitted function — the next slide

Two Ways to Generalise

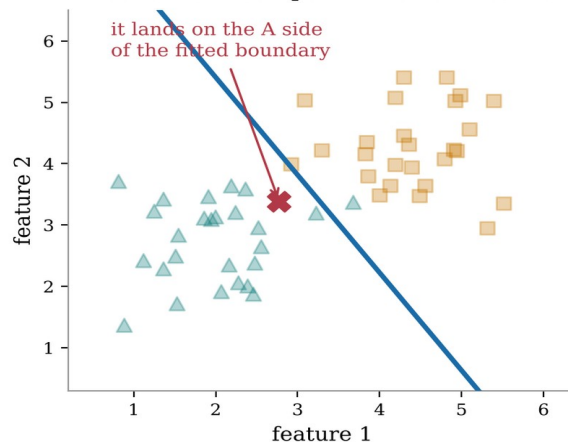
Géron Ch 1, Instance-Based vs Model-Based — what the learner keeps

- k-nearest neighbours keeps the training set and generalises by similarity. It shines on small, shifting data, but the whole set ships to production and every prediction is a search
- A model-based learner fits a rule and keeps only its parameters, so prediction is cheap and the examples can be discarded. This course is almost entirely model-based

Instance-based: compared with what you kept



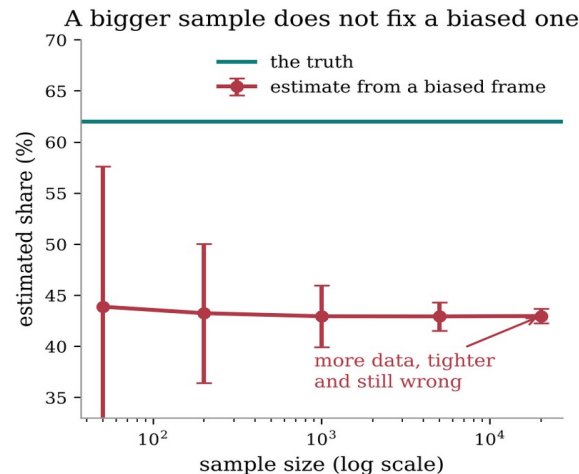
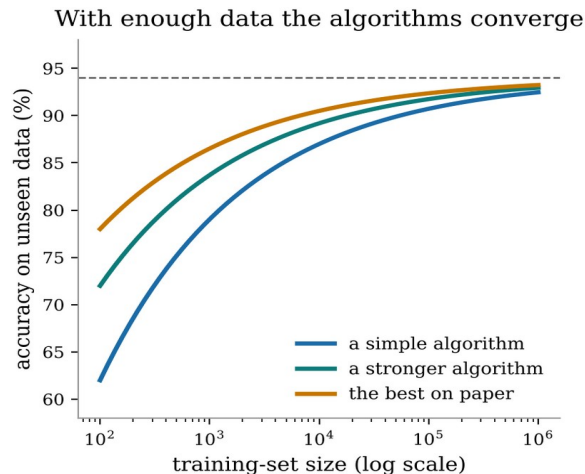
Model-based: compared with a fitted rule



When the Data Is Wrong

Géron Ch 1, Main Challenges — bad data before bad algorithms

- Too little data: even simple problems want thousands of examples, and past a point very different algorithms perform almost identically — so building the corpus can beat refining the algorithm
- Nonrepresentative data: when the sampling frame is wrong, a larger sample is only a tighter estimate of the wrong number. Sampling bias and nonresponse bias are not cured by volume



Features Make or Break It

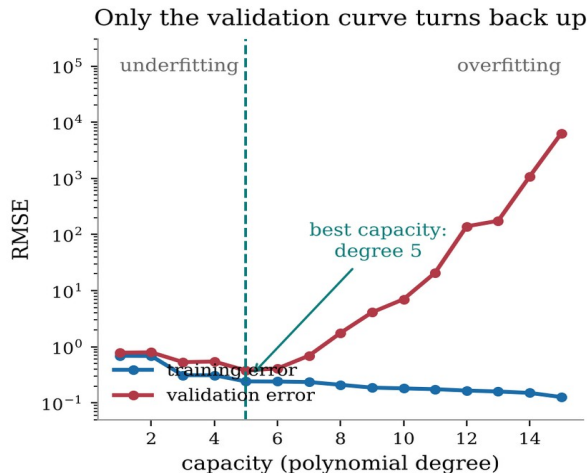
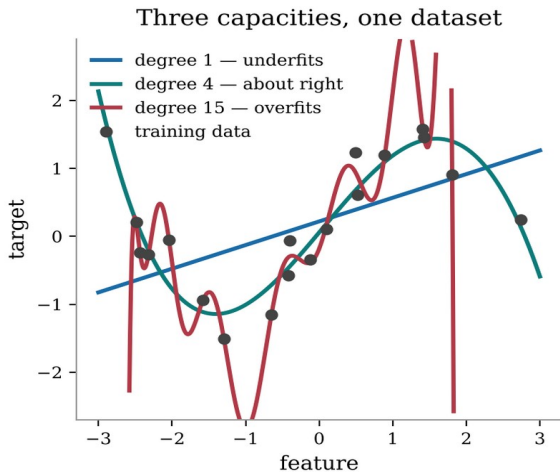
Géron Ch 1, Irrelevant Features — garbage in, garbage out

- Poor-quality data first: errors, outliers and missing values are worth real time to clean, which is part of why most of a project is spent on data and not on the model
- A model can only learn if the data carries enough relevant features and not too many irrelevant ones. Feature engineering is the work of getting there
- Three moves: selection, keeping what is useful; extraction, combining features into a better one — dimensionality reduction is exactly this (Chapter 7); and creating new features by gathering new data

Capacity and Overfitting

Part I companion (PDF), §1.2 — too little flexibility, and too much

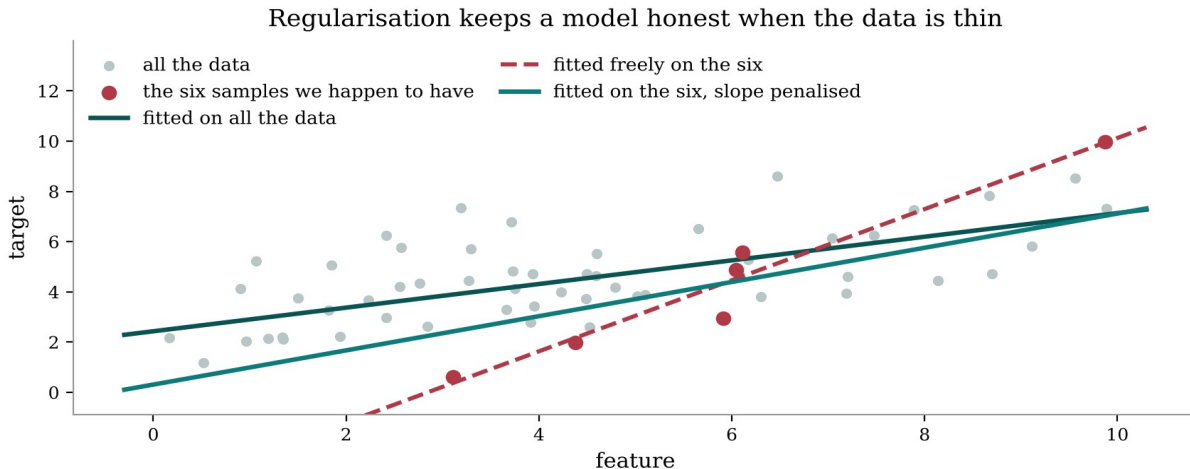
- Underfitting is too little capacity to capture the signal, so training and validation error are both high. Overfitting is enough capacity to memorise the noise, so training error falls while validation error climbs
- The diagnosis needs two numbers, not one: training error alone cannot tell the two apart, which is why the validation curve is the one that matters



Hyperparameters

Part I companion (PDF), §1.2 — the knobs you set rather than learn

- Regularisation constrains the parameters so the model cannot contort too freely. Penalise the slope and a fit on six samples stops chasing them — Chapter 4 makes this concrete as Ridge and Lasso
- A hyperparameter is any knob fixed before training rather than fitted during it — a regularisation strength, a tree depth, a learning rate — and it is tuned on validation data, never on the test set

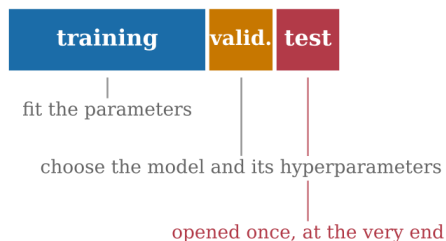


Honest Validation

Part I companion (PDF), §1.3 — hold out, or rotate

- We care about error on unseen data, so a test set is split off and frozen; model and hyperparameter choices are made on a separate validation set
- With few samples one validation split both wastes data and is noisy. k -fold cross-validation rotates the validation fold k times and averages, so every sample validates exactly once

Split once, early, and freeze the test set



or rotate the validation fold k times

and average the k scores

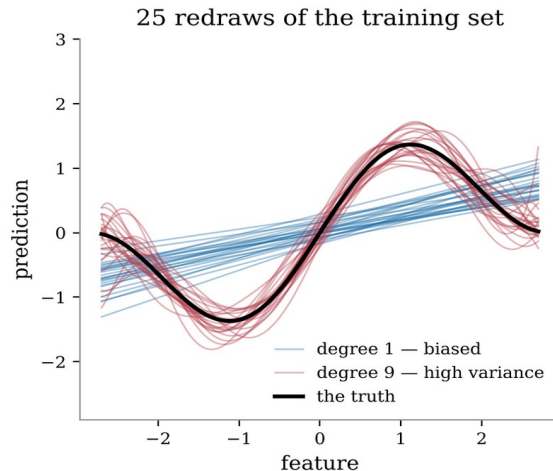
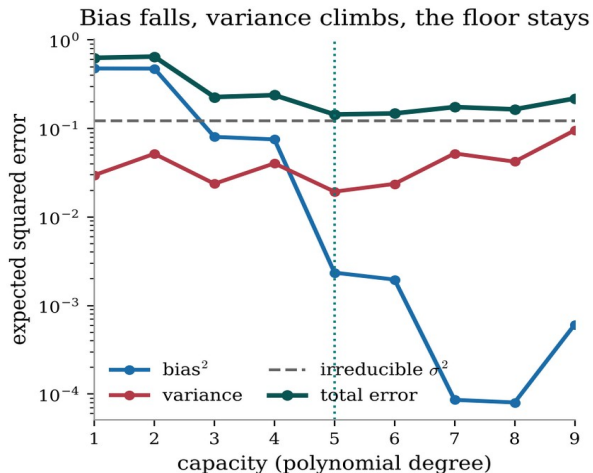


every sample validates exactly once — the small-dataset answer

Bias and Variance

Part I companion (PDF), §1.3 — one identity behind the whole part

- With $y = f(x) + \varepsilon$, the expected squared error splits into exactly three non-negative pieces — bias^2 , variance and the noise σ^2 . The cross terms vanish because ε is independent of the fit
- More capacity lowers bias and raises variance; less does the reverse. σ^2 is a floor no model beats, and lowering it is a measurement question for the lab, not a modelling one



No Free Lunch

Part I companion (PDF), §1.3 — why the assumptions are the real choice

- Averaged over all possible problems, no learning algorithm beats any other. An algorithm is good only because its assumptions match the problems we actually face
- So model selection is really assumption selection. That is why every chapter in Part I states its model's assumptions, and why we will keep asking what a method takes for granted
- The practical reading: knowing that a tree splits on one feature at a time, or that k-means assumes round clusters of similar size, is worth more than memorising either API

Let's practice — Chapter 1

[https://github.com/ageron/handson-mlp/blob/main/
01_the_machine_learning_landscape.ipynb](https://github.com/ageron/handson-mlp/blob/main/01_the_machine_learning_landscape.ipynb)

[Open Chapter 1 in Google Colab →](#)

Session 9



Session 9 — Agenda

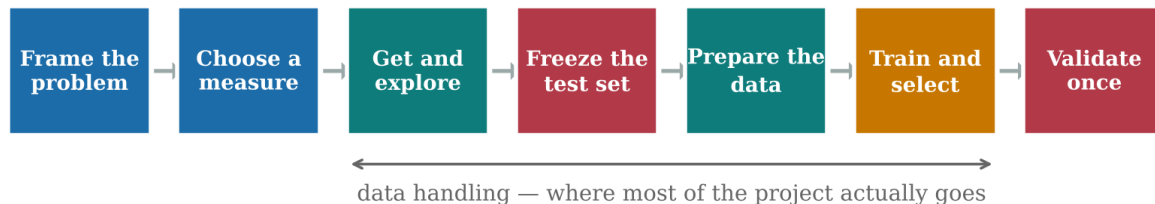
- Géron Chapter 2: the workflow a real project actually follows
 - framing, performance measures, the test set, exploring the data, preparing it, selecting a model, and keeping it alive
- Which error norm you report, and why it changes which model wins
- Why a random test split fails on rare outcomes, and what data snooping costs you
- Lab — Géron's Chapter 2 notebook: the California housing project, end to end
- Miniproject 1 released

The Workflow

Part I companion (PDF), §2 — seven steps, and modelling is one of them

- Frame the problem, choose a performance measure, get and explore the data, freeze a test set, prepare the data, train and select a model, validate once
- Framing comes first: what exactly is predicted, what the downstream system does with it, and what a simple baseline already achieves. Check those assumptions with whoever consumes the output
- The modelling is a small slice and data handling dominates. That ratio is not a failing of the field — it is what the job is. Every later chapter slots a concrete model into this same skeleton

The modelling is one box out of seven



Rows, Features, Labels

Géron Ch 2 — the notation every formula from here uses

- One row is one sample, written as the vector $\mathbf{x}^{(i)}$; its target is $y^{(i)}$. Stack the rows and you have the design matrix X of Module 2, with y collecting the targets
- Features come in kinds that need different handling: numerical ones get scaled, categorical ones get encoded, and the label is touched by neither

one sample $(\mathbf{x}^{(i)})^T \rightarrow$

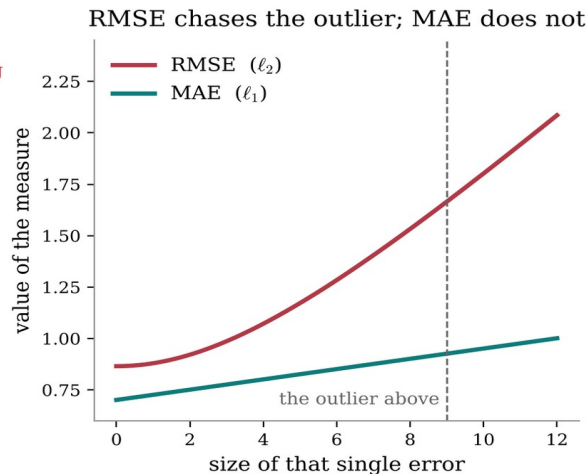
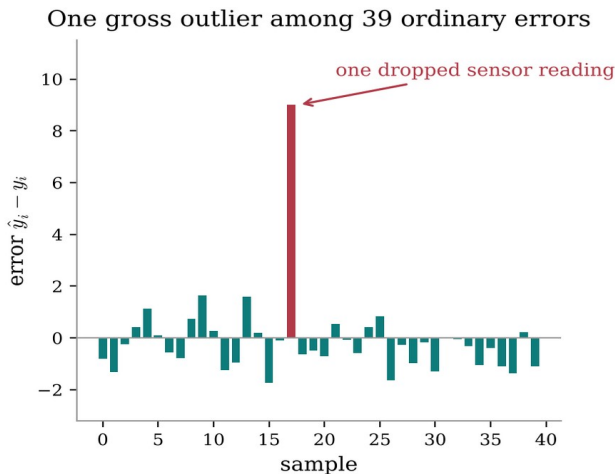
numerical features			categorical	label $y^{(i)}$
age	assay_1	assay_2	stage	biomarker
58	12.4	0.81	II	431
47	9.7	0.64	I	298
71	15.2	0.93	III	612
63	11.8	0.77	II	455

X stacks the samples as rows; y collects their targets. Every formula from here on is written in these symbols.

RMSE and MAE

Part I companion (PDF), §2.1 — ℓ_2 and ℓ_1 on the error vector

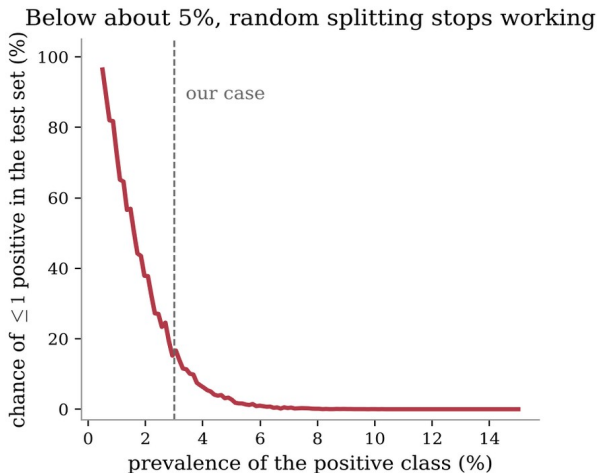
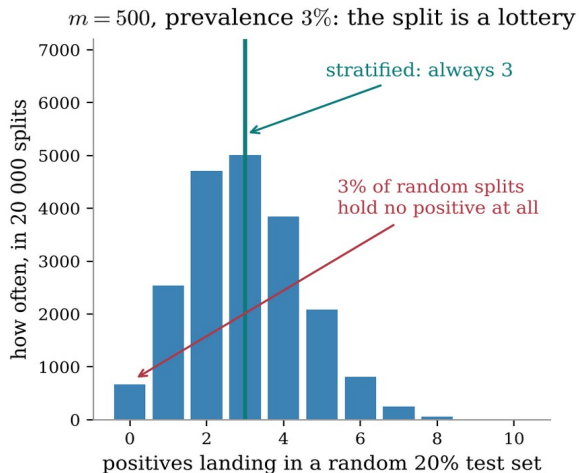
- RMSE is the ℓ_2 norm of the error vector and MAE the ℓ_1 norm — the two norms of Module 2, now applied to $\hat{y} - y$
- Squaring makes RMSE penalise large mistakes far more heavily, so it is right when big errors are especially costly and outliers are few; MAE is the robust choice. What you report and what you optimise need not be the same



A Trustworthy Test Set

Part I companion (PDF), §2.2 — split once, then stratify

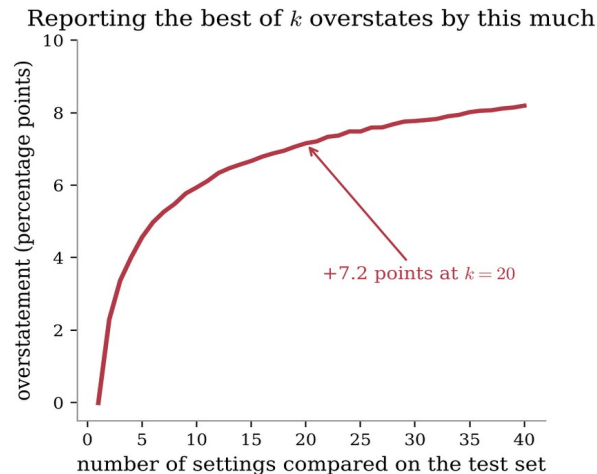
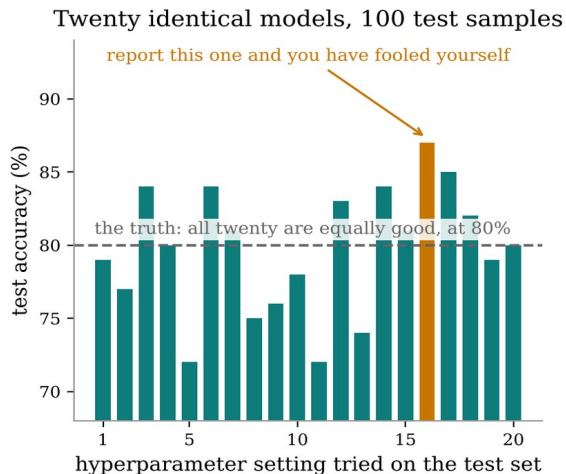
- Split the test set off once, early, and freeze it. Pure random splitting is fine for large data and dangerous for small or skewed data; stratified sampling partitions by an important attribute — age band, disease stage, outcome — and samples within each stratum
- At 3% prevalence in 500 samples a random 20% test set holds three positives on average and sometimes none at all. Stratifying is not a refinement here, it is a necessity



Data Snooping

Part I companion (PDF), §2.2 — what looking costs you

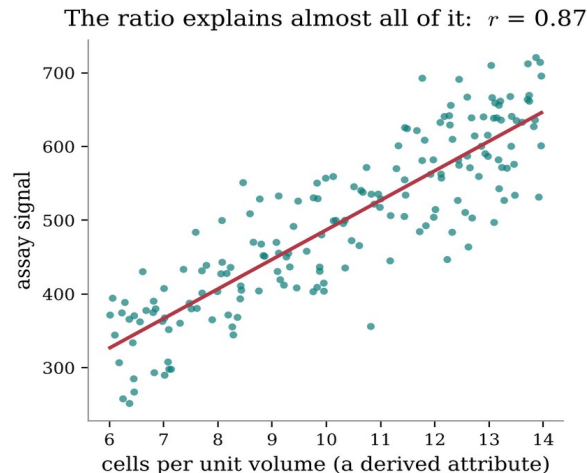
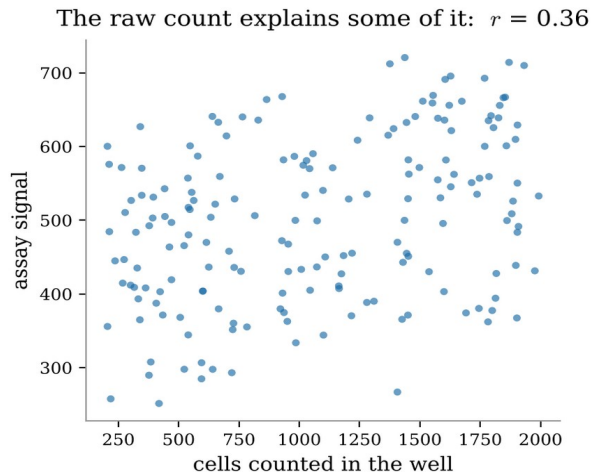
- Every look at the test set, even an informal one, lets patterns leak into your choices, and the estimate stops being an estimate of future performance
- Compare twenty equally good settings on a 100-sample test set and the best scores about seven points above the truth, purely from noise. The fix is procedural: choose on validation data, touch the test set once, report that number whatever it says



Explore and Visualise

Géron Ch 2, Explore and Visualize — look before you model

- Work on a copy of the training set only, and look: histograms for scale and skew, the correlation coefficient for linear association, a scatter matrix for the few attributes that matter
- Experiment with attribute combinations. A raw count can be weakly predictive while the same count per unit volume is strongly predictive — the derived attribute is what the process actually depends on



Preparing the Data

Part I companion (PDF), §2.3 — impute, encode, scale

- Impute missing values — the median for skewed clinical variables, because the mean follows the tail. One-hot encode unordered categories such as blood type, so no false ordering is invented
- Scale the features: standardising to zero mean and unit variance is the usual default, min–max maps to [0, 1] but is more exposed to outliers. Heavy-tailed features are logged or bucketed first

raw			imputed			encoded + scaled				
age	assay	stage	age	assay	stage	age	assay	I	II	III
58	12.4	II	58	12.4	II	0.0	-0.7	0	1	0
47	—	I	47	12.4	I	-1.1	-0.7	1	0	0
71	15.2	III	71	15.2	III	1.1	1.4	0	0	1

median

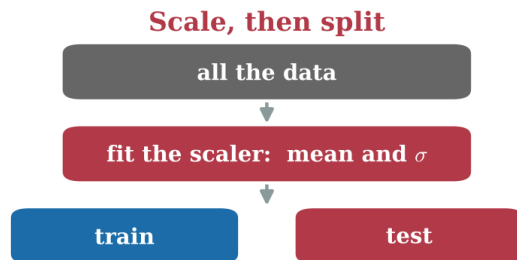
one-hot
+ standardise

only the features are transformed — and every fitted value here comes from the training set alone

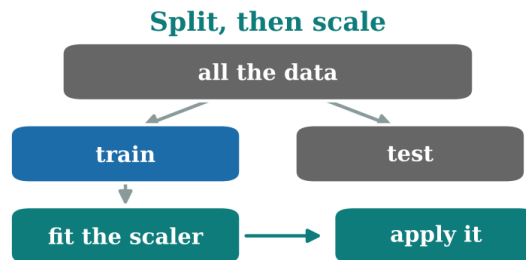
Fit on Training Only

Part I companion (PDF), §2.3 — the leak hiding in your preprocessing

- Estimate the mean and variance, or the min and max, on the training set alone; store them, and apply the stored values to validation and test
- Computing them on the whole dataset leaks test information into training. A pipeline chains imputation, encoding and scaling so the same fitted transformation is reused everywhere, and the leak becomes hard to commit



the test set helped choose the numbers
that transform the training set
the score comes out optimistic

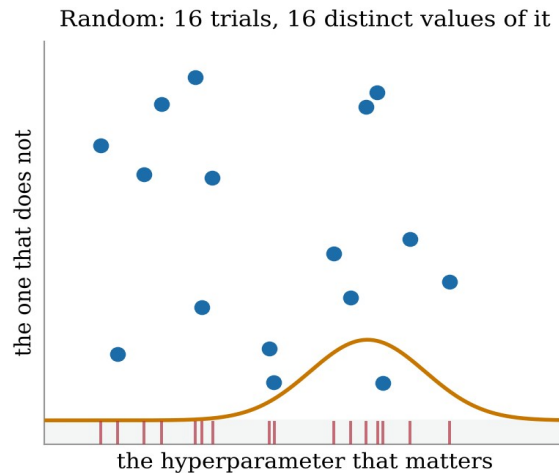
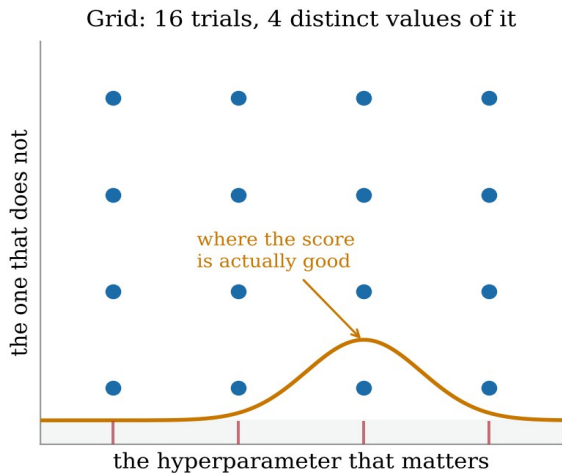


the stored mean and σ travel one way only,
from train to test
the score means something

Selecting a Model

Part I companion (PDF), §2.4 — the trade-off, now operational

- Train a few candidates, compare them by cross-validation, and only then estimate final performance once on the test set. High training error means underfitting; low training with high validation error means overfitting
- Hyperparameters are searched by grid or randomised search over cross-validated scores. Randomised search wins when the grid is large and most of its dimensions do not matter



A Biomedical Framing

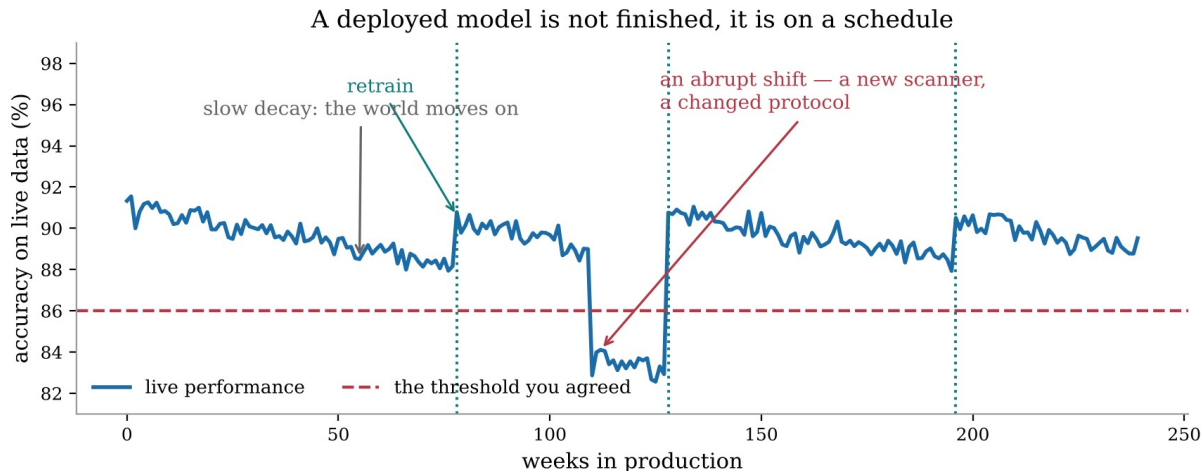
Part I companion (PDF), §2.4 — the skeleton on a real problem

- Predicting a continuous biomarker from 30 assay features: frame it as regression and choose RMSE, because large errors are clinically costly
- Stratify the test split by disease stage so every stage is represented, standardise on the training fold only, and cross-validate a linear model against a tree ensemble
- Report a single test RMSE with a rough confidence interval. Every chapter from here slots a different model into exactly this skeleton

Launch, Monitor, Maintain

Géron Ch 2, Launch, Monitor and Maintain — it does not end at the test score

- Deploying means saving the whole fitted pipeline, not just the model, so new data is transformed exactly as the training data was
- Models decay because the world moves on. Monitor live performance against a threshold you agreed in advance, plan the retraining, and keep every model and dataset version so you can roll back



Key Concepts So Far

Sessions 8 and 9 — what to carry into Chapter 3

- Kinds of variable: numerical and categorical features, and the label. Which one a column is decides how it is cleaned, encoded and scaled
- The three-way split and the discipline that goes with it: fit on training, choose on validation, open the test set once. Stratify when the data are small or skewed
- Capacity, overfitting and the bias–variance trade-off; and the rule that every fitted transformation is learned on the training set alone

Let's practice — Chapter 2

https://github.com/ageron/handson-mlp/blob/main/02_end_to_end_machine_learning_project.ipynb

[Open Chapter 2 in Google Colab](#) →

Before Next Session

- Work both Géron notebooks through to the end. Chapter 2's is the long one, and it is the one Miniproject 1 leans on
- Re-read the Part I companion §1.2–1.3 and §2.1–2.4: the bias–variance identity and the leakage rules are examinable
- Session 10 preview: Classification — the confusion matrix, precision and recall, ROC and AUC, and why accuracy lies on imbalanced data

Q & A



End of Sessions 8 & 9

Computing, Robotics and Bionics

Sergio Sama Rubio

Sant Cugat del Vallès, 5 & 8-Oct-2026